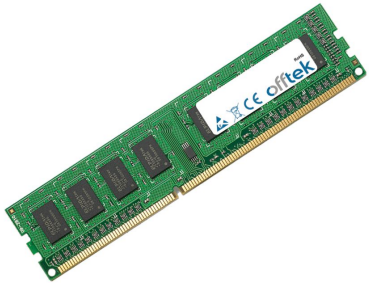
Computer Engineering, not Computer Science


intel
Core™ i7

Computer Architecture

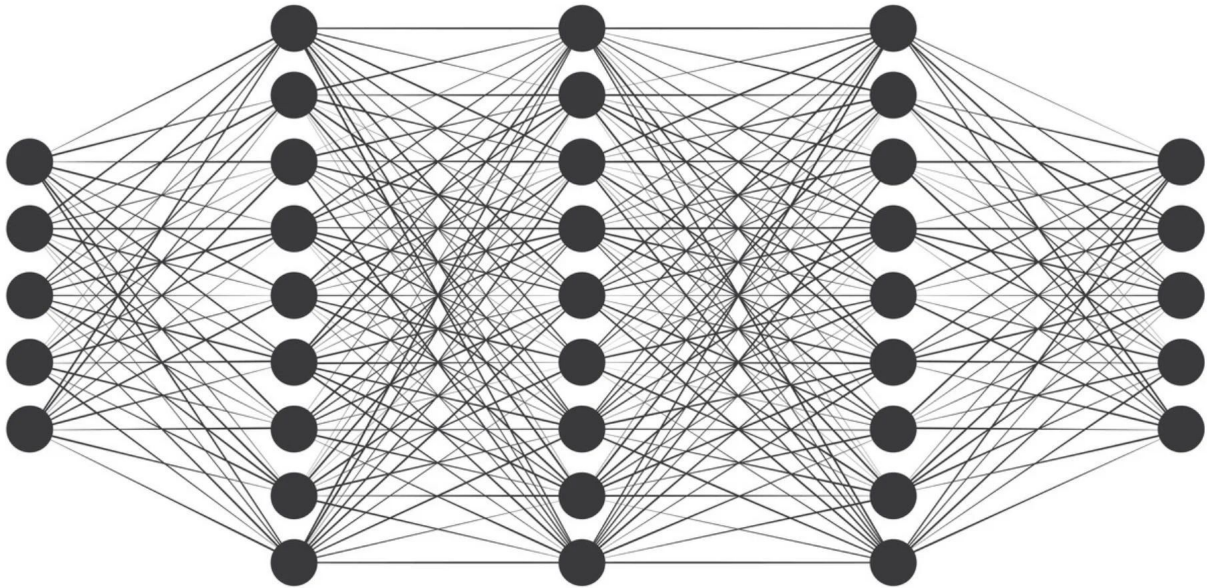



ChatGPT

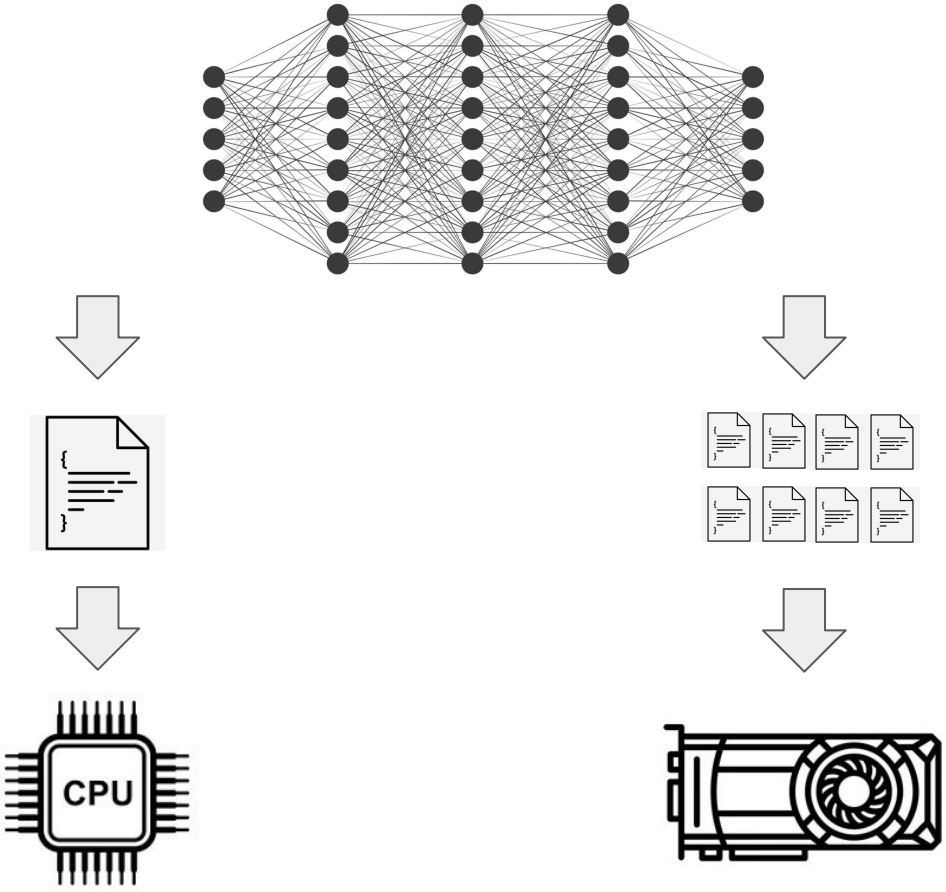

Gemini

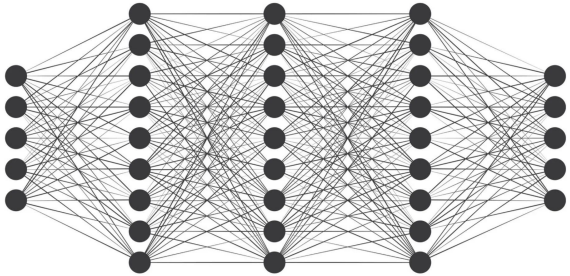

Claude

Artificial Neural Networks

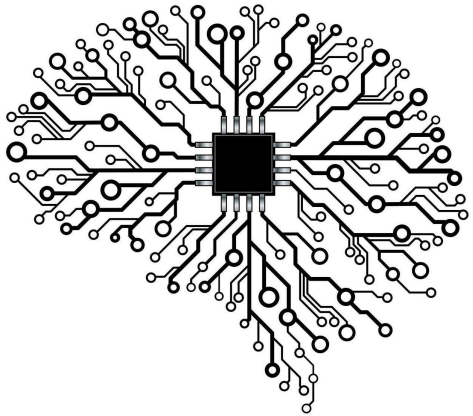


Deep Learning and Big Data brought the explosion of AI!


{
}
CPU

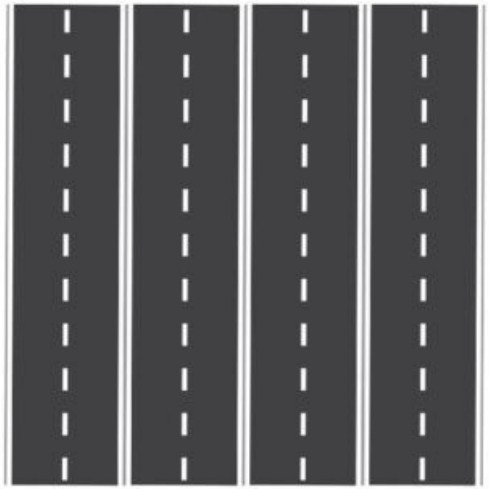


Neuromorphic Computing



- Brain Like
- Brings memory and compute elements closer together

CPU

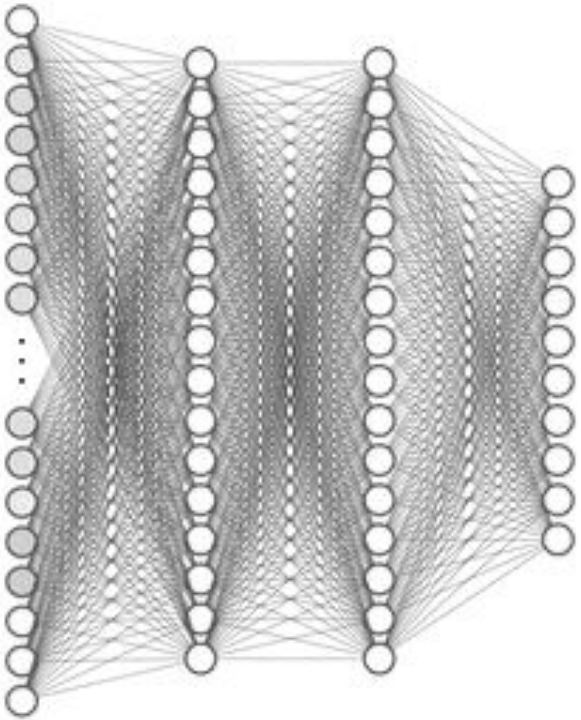


GPU

Neuromorphic Chip

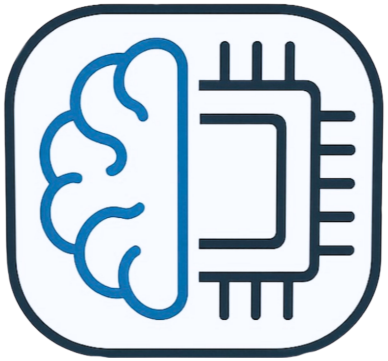
Spiking Neural Networks

- Sparsity = energy and computationally efficient



PeraMorphIQ

PeraCom Neuromorphic Research Group

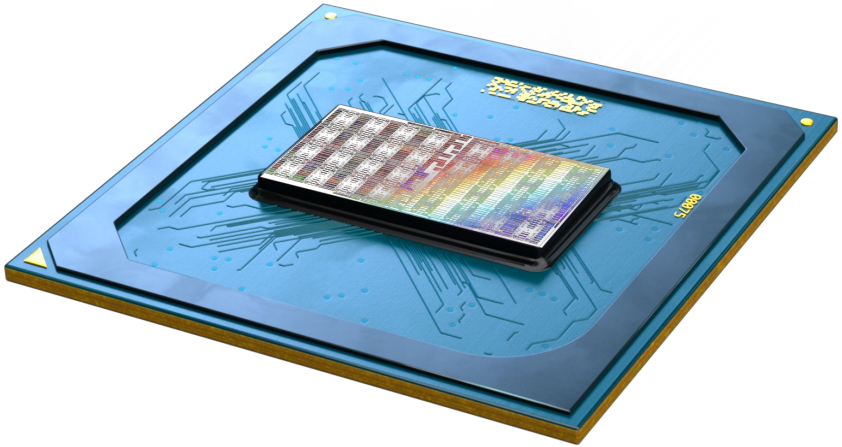


Supervisors

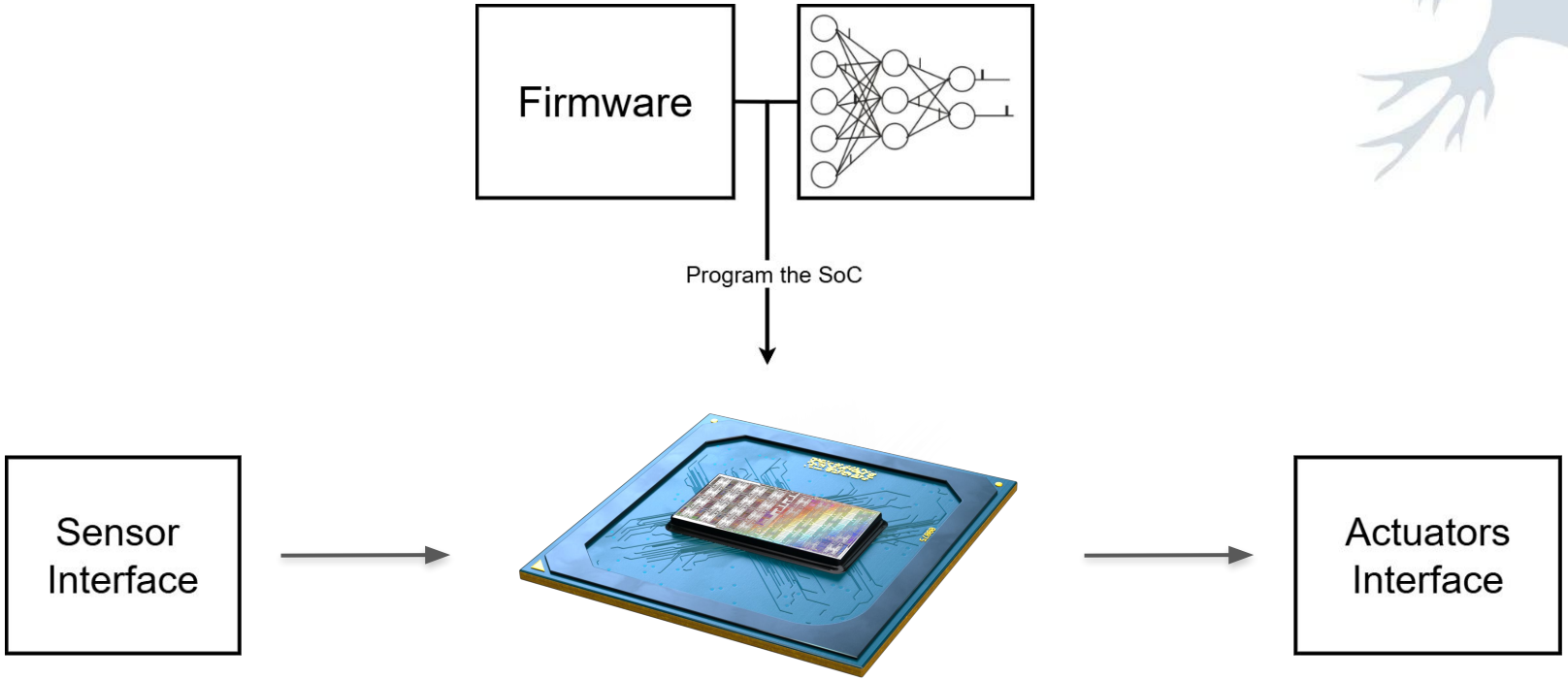
Dr. Isuru Nawinne

Prof. Roshan Ragel

What we are doing...



SNAP-V: Spiking Neural Accelerated RISC-V Processor


Firmware
Program the SoC
Sensor Interface
Actuators Interface

SNAP-V

Low Power Edge Systems with AI

Experience

neuromorphic architecture

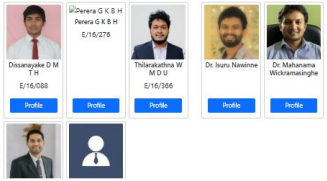
Created: 15-07-2022 Forks: 4 Watchers: 10 Stars: 10

Description

Neuromorphic architectures are hardware architectures that use the biologically inspired neural functions as the basis of operation. Information processing based on spiking neuron architectures have caught considerable attention in recent years due to its low power consumption compared to traditional artificial neural networks. In this project, as the first stage, we are implementing parallel multiple processing elements based on RISC-V architecture to represent biological neurons. Single neurons can be implemented as a single processor with local memory access or since the spike time of biological neurons is in the millisecond order multiple neurons can be virtualized to a single processor. At the second stage of the process, we are expecting to design encoders and decoders to benchmark the architecture by solving classical machine learning problems.

Project Repository | Project Page

Team / Supervisors



Neuromorphic NoC Architecture for SNNs

Created: 19-05-2023 Forks: 9 Watchers: 22 Stars: 22

Description

This project aims to develop a novel neuromorphic NoC architecture based on RISC-V ISA to support spiking neural network applications, and test it on FPGA.

Project Repository | Project Page

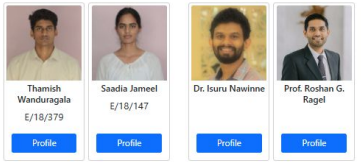
Team / Supervisors

Neuromorphic NoC Architecture for SNNs

Created: 07-01-2024 Forks: 5 Watchers: 7 Stars: 7

Project Repository | Project Page

Team / Supervisors



E19 - SNAP -V: A RISC -V SoC with Configurable Neuromorphic Acceleration for Small -Scale Spiking Neural Networks

Computer Science > Hardware Architecture

[Submitted on 12 Mar 2026]

SNAP-V: A RISC-V SoC with Configurable Neuromorphic Acceleration for Small-Scale Spiking Neural Networks

Kanishka Gunawardana, Sanka Peeris, Kavishka Rambukwella, Thamish Wanduragala, Saadia Jameel, Roshan Ragel, Isuru Nawinne

Spiking Neural Networks (SNNs) have gained significant attention in edge computing due to their low power consumption and computational efficiency. However, existing implementations either use conventional System on Chip (SoC) architectures that suffer from memory-processor bottlenecks, or large-scale neuromorphic hardware that is inefficient and wasteful for small-scale SNN applications. This work presents SNAP-V, a RISC-V-based neuromorphic SoC with two accelerator variants: Cerebra-S (bus-based) and Cerebra-H (Network-on-Chip (NoC)-based) which are optimized for small-scale SNN inference, integrating a RISC-V core for management tasks, with both accelerators featuring parallel processing nodes and distributed memory. Experimental results show close agreement between software and hardware inference, with an average accuracy deviation of 2.62% across multiple network configurations, and an average synaptic energy of 1.05 pJ per synaptic operation (SOP) in 45 nm CMOS technology. These results show that the proposed solution enables accurate, energy-efficient SNN inference suitable for real-time edge applications.

Comments: 12 pages, 5 figures, 5 tables
Subjects: **Hardware Architecture (cs.AR)**; Neural and Evolutionary Computing (cs.NE)
Cite as: arXiv:2603.11939 **[cs.AR]**
(or arXiv:2603.11939v1 **[cs.AR]** for this version)
https://doi.org/10.48550/arXiv.2603.11939

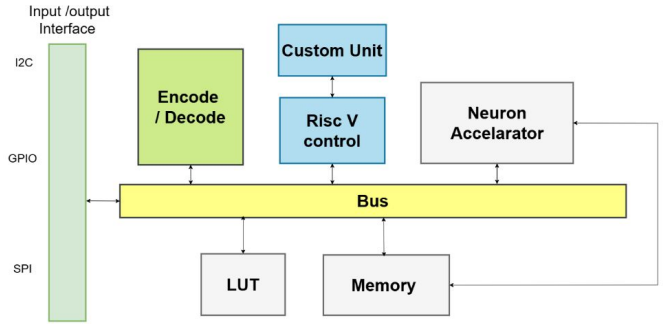
Submission history

From: Kanishka Gunawardana [view email]
[v1] Thu, 12 Mar 2026 13:52:44 UTC (395 KB)

E20 - On-Chip Offline Neuromorphic Computing

Three-State On-Chip Pipeline

The full on-chip learning loop operates as three sequential states per training sample:



Team / Supervisors

Methodology

Neuron Model

The system uses the **Leaky Integrate-and-Fire (LIF)** neuron model. At each discrete timestep:

```
V_mem[t] = β × V_mem[t-1] + Σ(w_i × spike_i[t])
if V_mem[t] ≥ V_threshold:
    spike_out = 1,  V_mem = 0  (reset)
else:
    spike_out = 0
```

where $\beta = 192/256 \approx 0.75$ is the leak factor and all values are Q16.16 fixed-point.

Surrogate Gradient

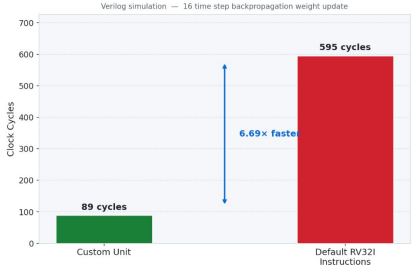
Because the Heaviside spiking function is non-differentiable, a **256-entry Q16.16 ROM LUT** stores pre-computed surrogate gradient values (smooth approximation of the sigmoid derivative). The LUT index is derived from the neuron's membrane potential `V_mem`.

Backpropagation

Weight updates use a surrogate-gradient backpropagation rule with momentum:

```
δ = ((error + 0.95 × δ_prev) × surrogate_grad × spike_status) >> 8
W' = W − (LR × δ) >> 8        (LR = 150/256 ≈ 0.586)
```

This computation is accelerated by the custom RISC-V backprop unit, which includes two PISO LIFO buffers (one for spike status, one for gradients) and a DMA-style Memory-to-LIFO Loader FSM.



Neuromorphic Accelerator: Execution Phase Breakdown

E20 - On-Chip Online Learning For Neuromorphic Hardware

Experiment Setup and Implementation

To evaluate the partitioned EONS architecture, a comprehensive hardware-software co-simulation methodology was established. Verilator was utilized to perform cycle-accurate co-simulations. The core event-driven components were implemented at the Register-Transfer Level (RTL) and seamlessly interfaced with the software-designated evolutionary components written natively in C++.

The accelerator was benchmarked using the standard MNIST handwritten digit dataset using a rate-coding scheme. To calculate fitness efficiently, the C++ Inference Controller dynamically samples a representative subset (10%) of the total dataset for manual testing. Over a defined number of generations, the C++ Cross-Over unit generates new network parameters that are continually written back to the hardware Neuron Arrays.

Accelerator Design Diagram

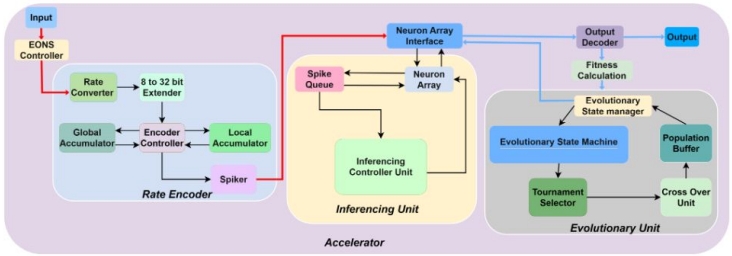


Figure 2: Detailed microarchitecture of the neuromorphic hardware accelerator.

Team / Supervisors

Bandara G.M.M.R.
E/20/034
Profile

Pathirage R.S.
E/20/280
Profile

Sriyarathna D.H.
E/20/385
Profile

Prof. Roshan G. Ragel
Profile

Dr. Isuru Nawinne
Profile

Methodology

The system operates using a Hardware-Software (HW/SW) co-design architecture to prevent overpopulating the silicon area with complex, non-critical control components. Computationally heavy genetic operations such as fitness calculation, tournament selection, and cross-over are partitioned into software.

Conversely, the high-throughput, event-driven data paths are implemented strictly in hardware. The hardware accelerator utilizes a Leaky Integrate-and-Fire (LIF) neuron model and consists of a Rate Encoder to convert real-valued inputs into spike trains, an Inference Unit (featuring a Network-on-Chip and banked neuron arrays), and an Evolutionary State Manager to handle the learning loops.

High-Level Design Diagram

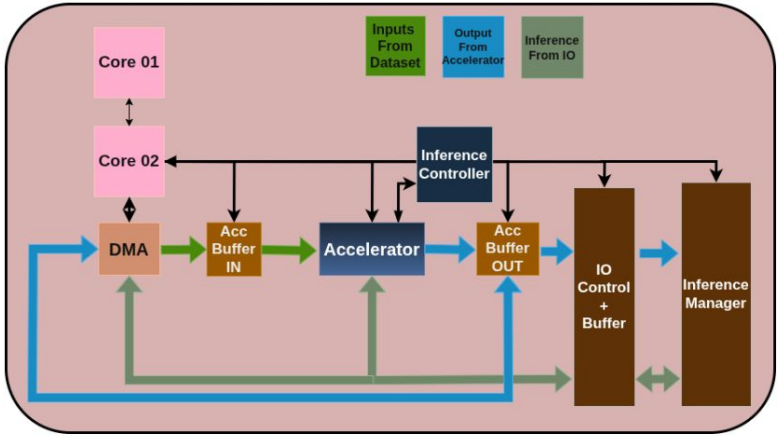
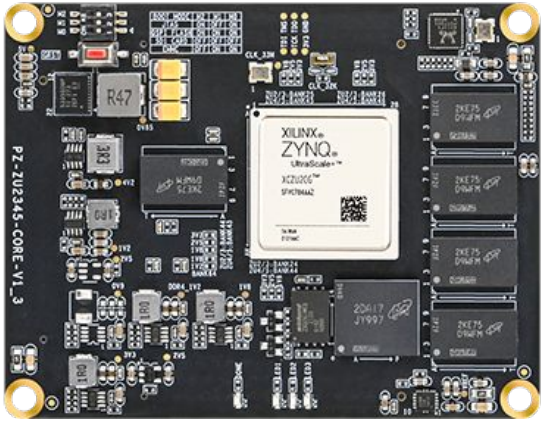


Figure 1: High-level architecture of the on-chip online learning neuromorphic accelerator.

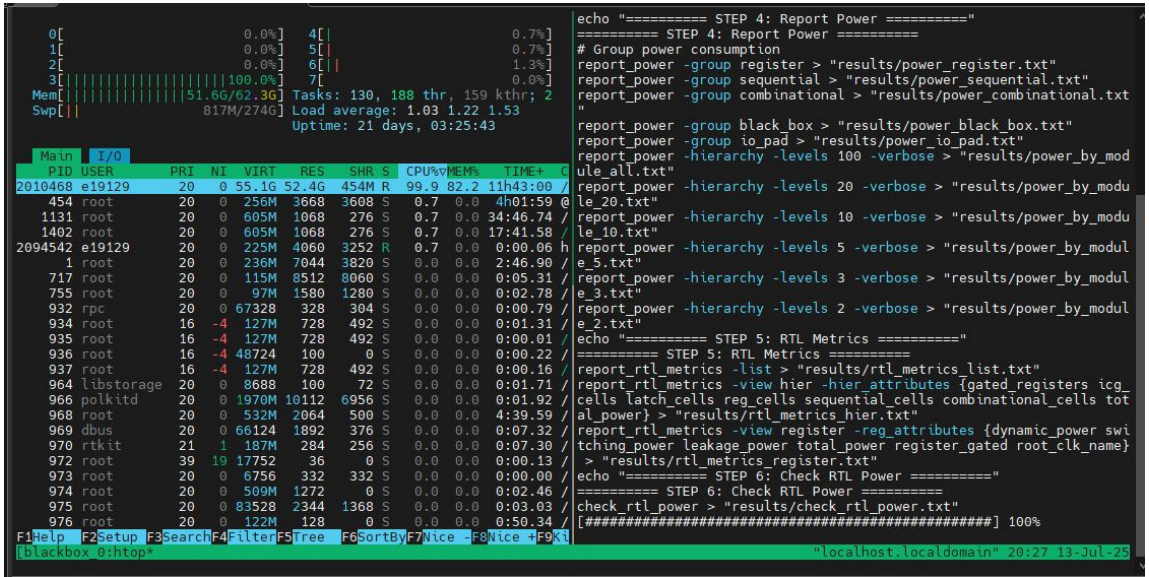
E21 -
Starting...

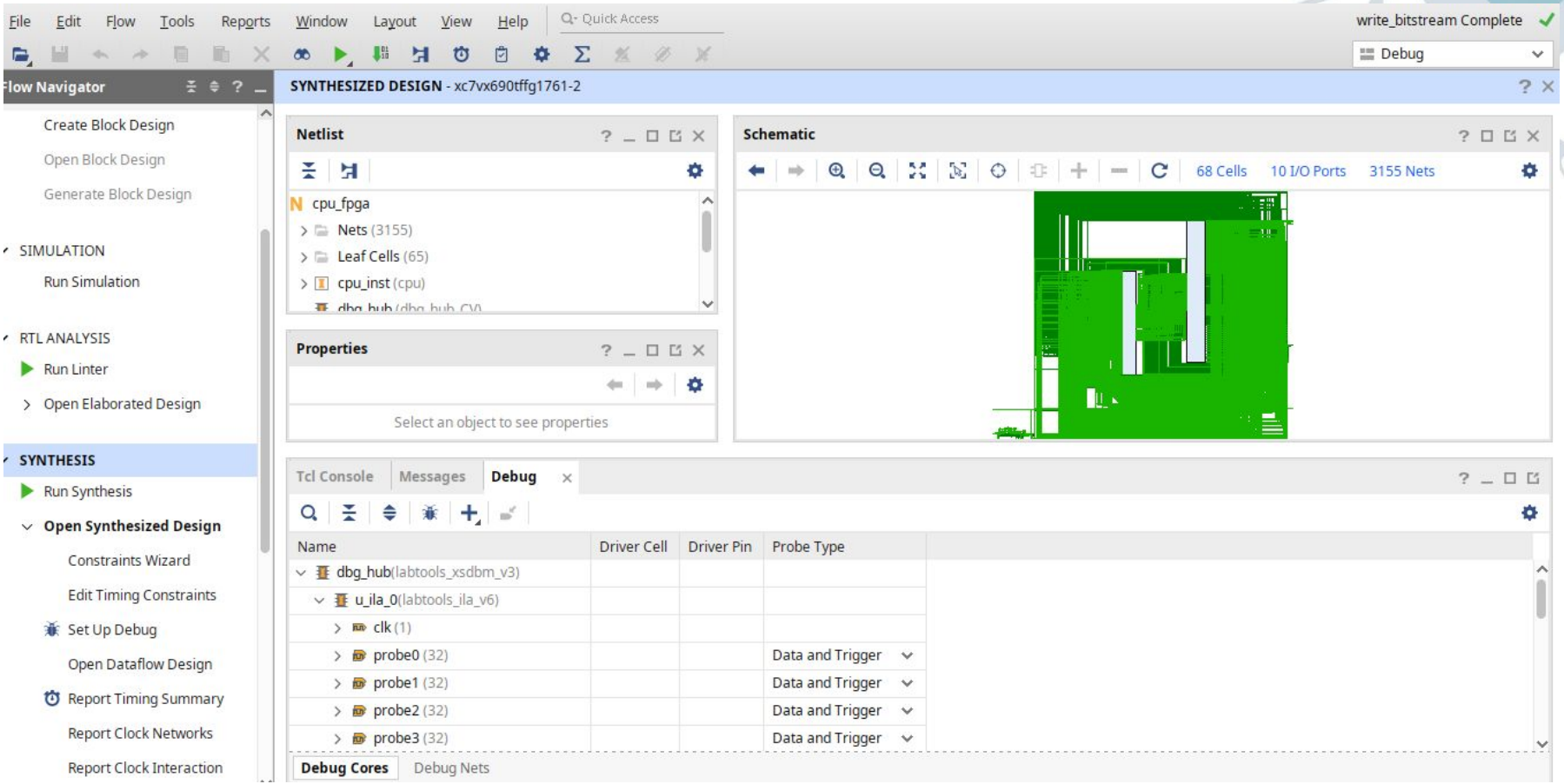
Servers and FPGAs

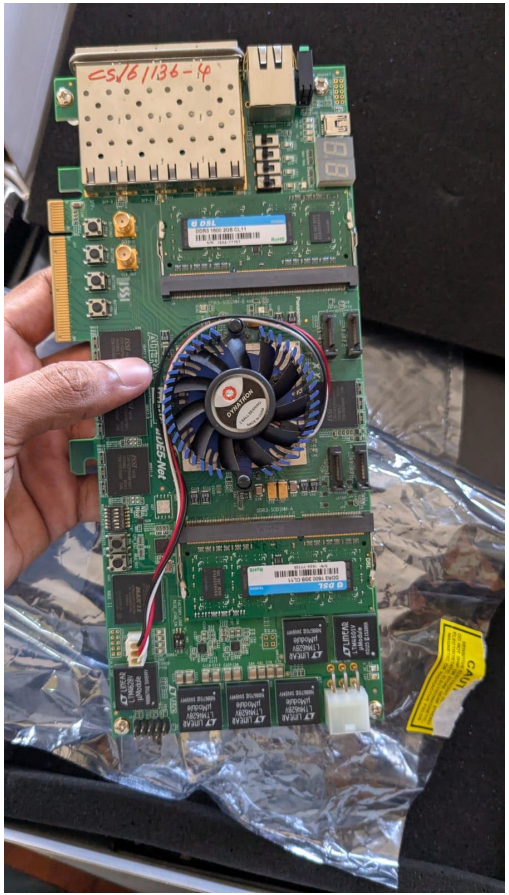


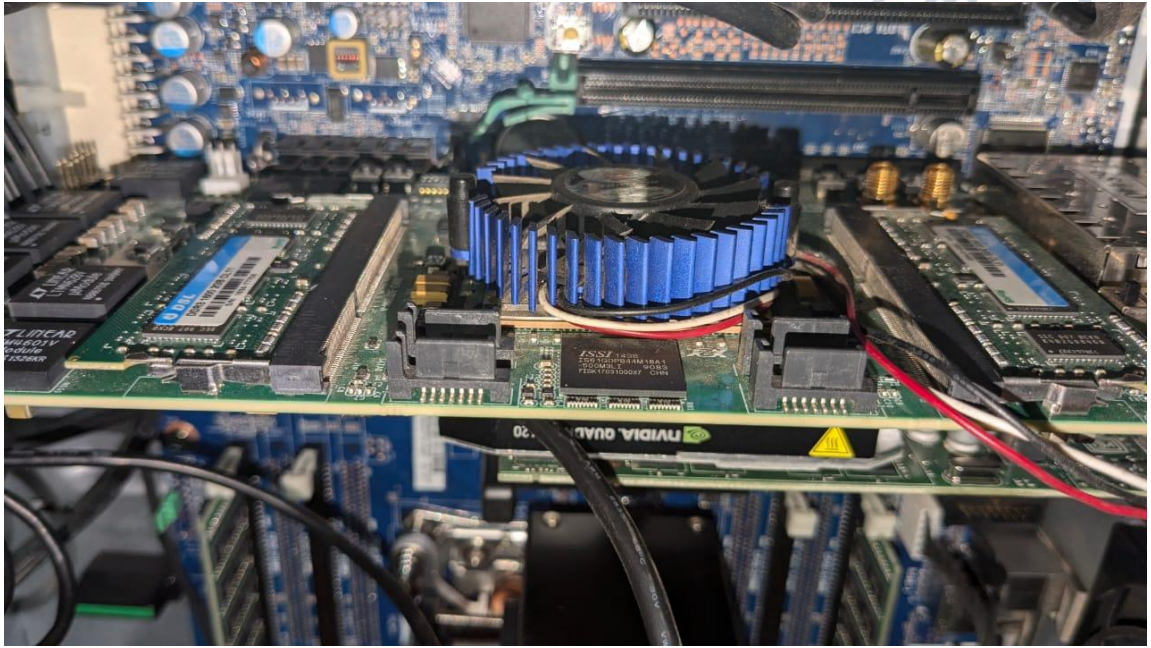
```
                        PrimePower

            Version W-2024.09-SP2 for linux64 - Nov 27, 2024

                Copyright (c) 1988 - 2024 Synopsys, Inc.
   This software and the associated documentation are proprietary to Synopsys,
 Inc. This software may only be used in accordance with the terms and conditions
 of a written license agreement with Synopsys, Inc. All other use, reproduction,
   or distribution of this software is strictly prohibited.  Licensed Products
     communicate with Synopsys servers for the purpose of providing software
    updates, detecting software piracy and verifying that customers are using
    Licensed Products in conformity with the applicable License Key for such
  Licensed Products. Synopsys will use information gathered in connection with
    this process to deliver software updates and pursue software pirates and
                              infringers.

 Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on
            Inclusivity and Diversity" (Refer to article 000036315 at
                        https://solvnetplus.synopsys.com)
set_app_var power_enable_rtl_analysis true
Information: Checked out license 'PrimePower-RTL' (PT-019)
```

SYNOPSYS®



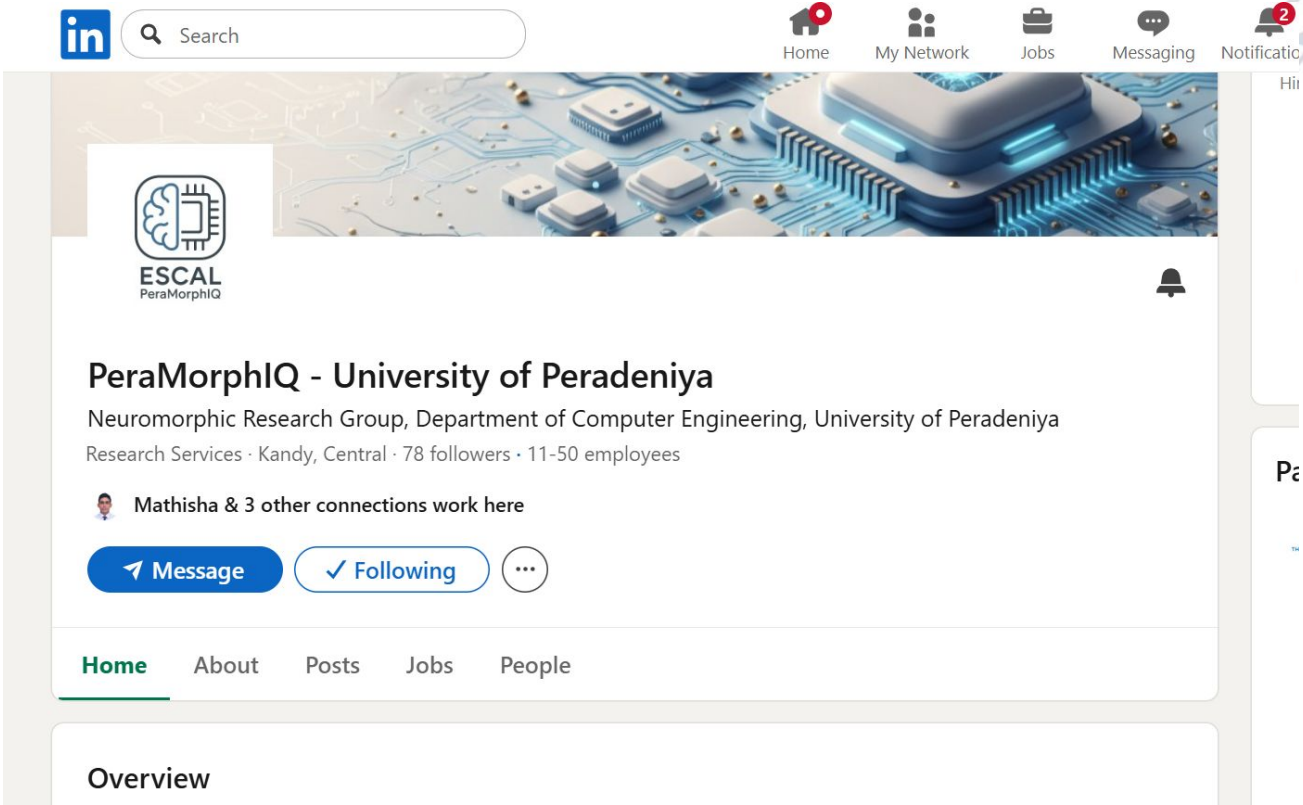

File Edit Flow Tools Reports Window Layout View Help
Quick Access
write_bitstream Complete
Debug
Flow Navigator
SYNTHESIZED DESIGN - xc7vx690tffg1761-2
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Run Linter
Open Elaborated Design
SYNTHESIS
Run Synthesis
Open Synthesized Design
Constraints Wizard
Edit Timing Constraints
Set Up Debug
Open Dataflow Design
Report Timing Summary
Report Clock Networks
Report Clock Interaction
Netlist
cpu_fpga
Nets (3155)
Leaf Cells (65)
cpu_inst (cpu)
Properties
Select an object to see properties
Schematic
68 Cells
10 I/O Ports
3155 Nets
Tcl Console
Messages
Debug
Name
Driver Cell
Driver Pin
Probe Type
dbg_hub(labtools_xsdbm_v3)
u_ila_0(labtools_ila_v6)
clk (1)
probe0 (32)
probe1 (32)
probe2 (32)
probe3 (32)
Data and Trigger
Debug Cores
Debug Nets


CS161136-4




University of Peradeniya
Neuromorphic NoC Accelerator Architecture for Spiking Neural Networks

https://www.linkedin.com/company/peramorphiq



If you're interested in:

Building Processors

Working with Computer Hardware

Contributing to the NEXT BIG THING

Let us know - you’re always welcome!!!